

NATS Queue Management Utilities – Technical Documentation

1. Project Overview

The NATS Queue Management Utilities is a .NET-based utility library and console application designed to simplify integration with the NATS messaging system. It enables developers to easily publish and subscribe to NATS subjects, supporting scalable, event-driven architectures.

2. Key Features

- Lightweight .NET integration with NATS
- Message publishing to NATS subjects
- Message subscription and real-time consumption
- Automatic connection handling
- Console-based debugging and logging
- Designed for future JetStream support

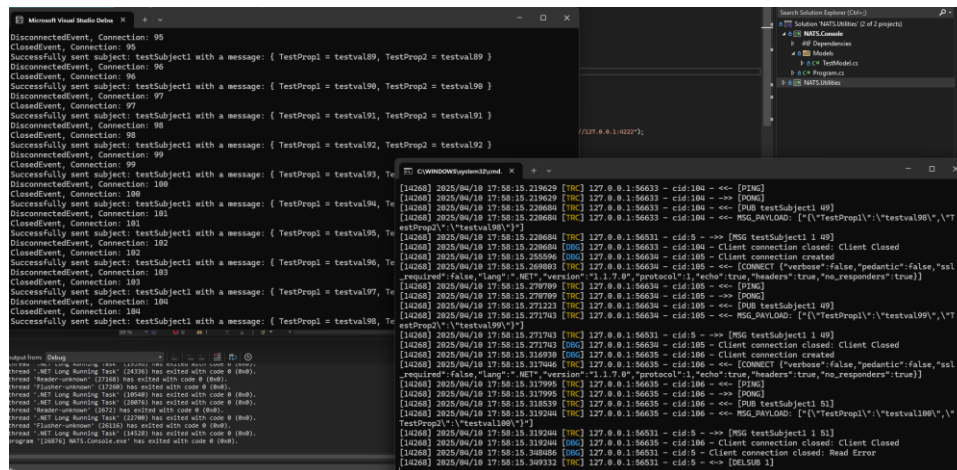
3. Technologies Used

- .NET Core / C#
- NATS Messaging System
- NATS .NET Client Library
- Visual Studio
- TCP/IP Networking

4. Solution Structure

The solution is composed of two primary projects:

- NATS.Console – A console application used to test publishing and subscribing
- NATS.Utilities – A reusable class library containing NATS connection and queue logic



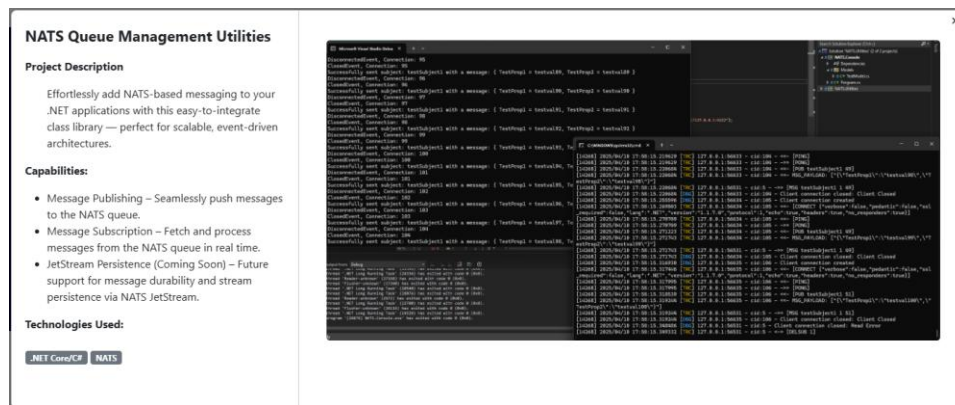
The screenshot displays the Visual Studio IDE with a solution named 'NATS1881818181' containing two projects: 'NATS.Console' and 'NATS.Utilities'. The 'NATS.Console' project is selected, and its output window shows a series of log messages. The messages indicate successful connections to NATS, message publishing to various subjects (e.g., 'testval09', 'testval90', 'testval91', 'testval92', 'testval93', 'testval94', 'testval95', 'testval96', 'testval97', 'testval98', 'testval99'), and message consumption from these subjects. The output also shows the state of the NATS connection, including 'Client connection created' and 'Client connection closed' events. The console output is formatted with timestamps, log levels, and message details.

5. Application Workflow

1. Establish connection to the NATS server
2. Publish messages to a configured subject
3. Subscribe to the subject and listen for incoming messages
4. Log connection, publish, and disconnect events
5. Gracefully close the connection after message processing

6. Message Publishing

The publishing component sends serialized messages to a specified NATS subject. Each message contains test properties (e.g., TestProp1, TestProp2) used to validate successful transmission.



7. Message Subscription

Subscribers listen to the configured subject in real time. Incoming messages are deserialized and logged to the console, verifying correct delivery and queue handling.

8. Connection Lifecycle Handling

The utility handles multiple connection states including:

- ConnectedEvent
- DisconnectedEvent
- ClosedEvent

These events are logged to assist with debugging and monitoring.

9. Logging & Debug Output

Verbose console output displays:

- Successful publish confirmations
- Message payloads
- Connection creation and termination
- Client disconnect and error events

This output is especially useful during development and testing.

10. Error Handling

The system gracefully handles client disconnects, read errors, and reconnections. All error events are logged without crashing the application.

11. Security Considerations

- Support for secured NATS endpoints (TLS)
- Credential-based authentication (if enabled)
- Controlled subject access
- Network firewall restrictions

12. Future Enhancements

- JetStream persistence and message durability
- Retry and acknowledgment handling
- Configuration via appsettings.json
- Centralized logging integration
- Metrics and monitoring support

13. Use Cases

- Event-driven microservices
- Background job processing
- Real-time notifications
- Distributed system communication
- Scalable queue-based architectures

14. Conclusion

The NATS Queue Management Utilities provides a clean and extensible foundation for integrating NATS messaging into .NET applications. Its modular design makes it suitable for both development testing and production-ready event-driven systems.